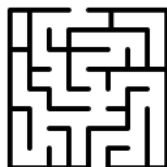
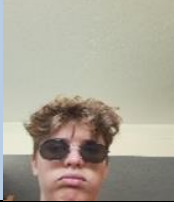


PI de Sistemas de Informação Distribuídos (25/26)

Licenciaturas em Engenharia Informática e Informática e Gestão de Empresas



<u>Grupo</u>	<u>13</u>
123278 Diogo Carvalho dccac@iscte-iul.pt	
123299 Igor Batista ilbao@iscte-iul.pt	
123294 Rodrigo Matias raams1@iscte-iul.pt	
105350 Tomás Évora tessa@iscte-iul.pt	
122628 Tomás Salvador tflsr@iscte-ul.pt	



123766
Vitor Pequito
vesfp@iscte-ul.pt



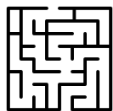
Grupo com quem trocam especificações: 16



Instruções

Estas instruções são de cumprimento obrigatório. Relatórios que não cumpram as indicações serão penalizados na nota final.

- Podem (e em várias situações será necessário) ser adicionadas novas páginas ao relatório, mas não podem ser removidas páginas. Se uma secção não for relevante, fica em branco, não pode ser removida;
- A paginação tem de ser sequencial e não ter falhas;
- O índice tem de estar atualizado.
- O grupo que inicia o documento (coluna à esquerda na folha de rosto) preenche apenas a parte inicial (até ao fim da secção secção 1). Este documento word vai ser colocado no moodle para que o outro grupo (à direita da folha de rosto) possa descarregar e continuar a preenche-lo (secção 2)



Índice

1	Especificação	5
1.1	Da Nuvem para o Mongo	5
1.2	Descrição Geral do Procedimento de Mongo Para Mysql	6
1.4	Tratamento de dados anómalos (valores de sensores errados)	8
1.5	Tratamento de outliers de ruído	9
1.6	Tratamento de Alertas de ruído	10
1.7	Tratamento de número de marsamis numa sala (obter pontuação).....	11
1.8	Especificação de Store Procedures SQL de apoio à migração e tratamento de dados 12	
1.9	Especificação de Triggers de apoio à migração e tratamento de dados.....	13
1.10	Modelo Relacional	14
1.11	Utilizadores Base de Dados Mysql.....	15
1.12	Procedimentos Manutenção da Aplicação.....	16
1.13	Eventos de suporte à aplicação (caso existam)	17
1.14	Consulta por HTML/PHP	18
2	Implementação	19
		19
2.1	Coleções a criar em cada uma das réplicas do Mongo	19
2.2	Descrição Geral do Procedimento de Mongo Para Mysql	20
2.3	Tratamento de dados anómalos (valores de sensores errados)	23
2.4	Tratamento de outliers de ruído	24
2.5	Tratamento de Alertas de Som.....	25
2.6	Tratamento de número de marsamis numa sala (obter pontuação).....	26
2.7	Implementação de Stored Procedures SQL de apoio à migração e tratamento de dados 27	
2.8	Implementação de Triggers	28
2.1	Modelo Relacional	29
2.2	Utilizadores Base de Dados Mysql.....	30
2.3	Procedimentos Manutenção da Aplicação.....	31
2.4	Eventos de suporte à aplicação (caso existam)	32



PI de Sistemas de Informação Distribuídos, 25/26

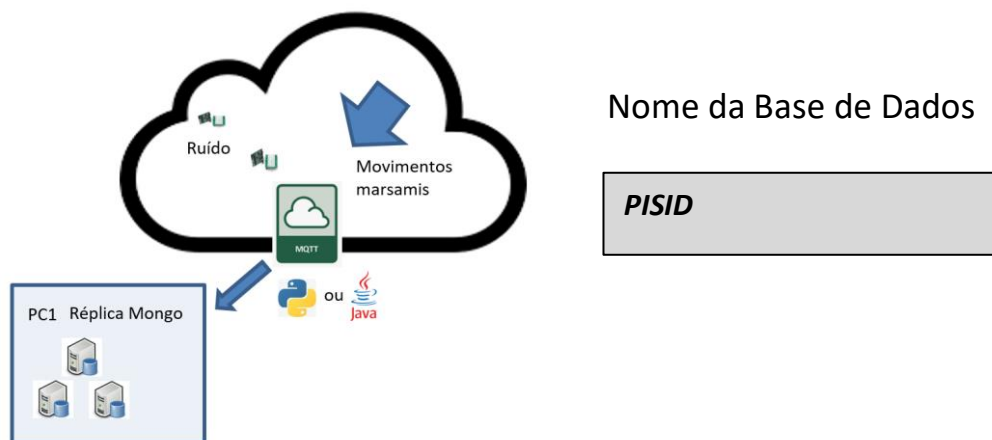
2.5	PrintScreen dos formulários HTML implementados	33
2.6	PrintScreen do formulários Android com dados	34
	Código de Triggers implementados	36
	Código Stored Procedures implementados	37



1 Especificação

Esta secção é onde o grupo que inicia o documento (coluna à esquerda na folha de rosto) coloca a especificação do que pretende implementar. Mais tarde pode implementar de outra maneira, mas aqui vão as primeiras ideias que serão avaliadas na primeira oral e que vão ser entregues a outro grupo para que analisem e vejam se aproveitam as vossas ideias.

1.1 Da Nuvem para o Mongo



Nome Coleção	O que armazena?
Temperature	Leitura do sensor da temperatura
Sound	Leitura do sensor do som
Motion	Cada vez que um marsami passa um corredor

Para cada coleção exemplifica um documento



Coleção : Temperature

```
{"_id": {  
  "$oid": "69b42c48800f112745b1d253"  
},  
"Player": 13,  
"Hour": "2026-03-13 15:24:56.590239",  
"Temperature": 15.15  
}
```

Coleção : Sound

```
{"_id": {  
  "$oid": "69b42c48800f112745b1d254"  
},  
"Player": 13,  
"Hour": "2026-03-13 15:24:56.636001",  
"Sound": 10.2  
}}
```

Coleção : Motion

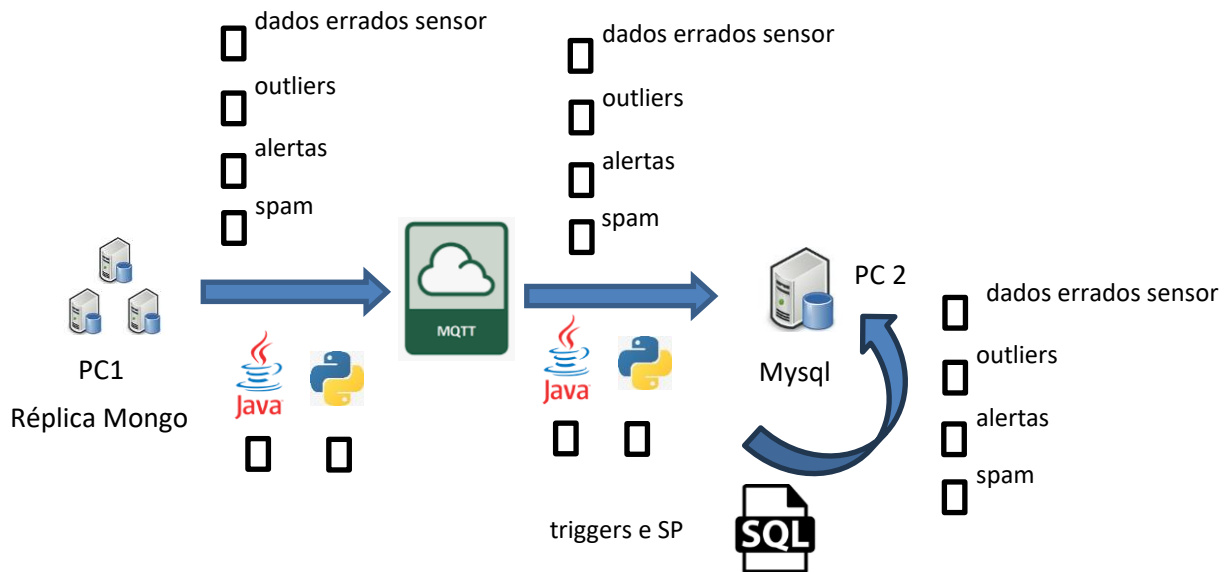
```
{"_id": {  
  "$oid": "69b42c27800f112745b1d234"  
},  
"Player": 13,  
"Marsami": 1,  
"RoomOrigin": 0,  
"RoomDestiny": 3,  
"Status": 1
```





1.2 Descrição Geral do Procedimento de Mongo Para Mysql

A passagem do Mongo para Mysql tem duas fases: a) enviar de Mongo para MQTT e b) receber de MQTT para Mysql. No diagrama deve ser indicado (nas check box) em qual das fases são programadas (em java e/ou python) as tarefas enumeradas (podem ser na fase a) b) ou ambas).



- Com que “periodicidade” (segundos) o programa vai buscar ao mongo: 1s
- Como garantem que não enviam duas vezes o mesmo documento do Mongo para o Mysql / MQTT (com base em datas ou booleano ou etc.):

Adicionar um campo booleano “migrated”, que fica true após o documento ter sido migrado.

- Pensam usar threads do Mongo para MQTT?: Sim e do MQTT para Mysql Sim?
- Quantas threads e/ou quantos maim em cada um dos passos?:
3_Threads
- No transporte MQTT que QOS vão utilizar? Porquê?
QOS_1, porque garante que a mensagem chega pelo menos uma vez, e o campo migrated vai evitar duplicados, coisa que o QOS_1 nao garant e.



Aqui podem desenvolver informação que considerem relevante relativo ao processo de migração, aspectos que não esteja refletido nas secções seguintes.



1.3 Tratamento de dados anómalos (valores de sensores errados)

Aqui devem explicar o que fazer caso se detectem valores “errados” (datas impossíveis, caracteres estranhos, etc..). Se recorrerem a triggers ou SP então indicam em secção mais adiante.

Marcar como anómalo, no python.

Situações- Som negativo; °C > 150? e °C < -50? (apenas exemplo estes valor podemos ver mais tarde concretamente); Som acima de 100db (o mesmo q os ultimos parenteses); Valores impossiveis.



1.4 Tratamento de outliers de ruído

Aqui devem explicar o que fazer caso se detectem "outliers" (valores fisicamente possíveis mas irrealistas, como por exemplo variações muito bruscas do ruído apenas num segundo). Se recorrerem a triggers ou SP então indicam em secção mais adiante.

Recorrer a trigger no caso da temperatura, se um valor subir ou descer 5°C num instante, considerar esse dado como outlier. o stor diz fazer python

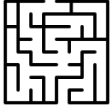
No caso do som, usar o python, se um valor subir, ou descer, drasticamente, 5db ou 10db, verificar os 2 valores seguintes, se o valor não se mantiver e voltar ao que estava antes do pico. Considerar outlier



1.5 Tratamento de Alertas de ruído e temperatura

Aqui devem explicar o que fazer caso se detectem situações alarmantes relativos ao som e temperatura. Se recorrerem a triggers ou SP então indicam em secção mais adiante. Qu situações despoletam os alertas, onde e como são armazenados. Explicar se existem mecanismos para evitar "spam" (demasiadas mensagens). É aconselhável recorrer a esquemas gráficos para explicar o mecanismo.

Alertas quando o som e a temperatura passam o valor máximo, e quando a temperatura passa o valor mínimo. Armazenados na BD no MySQL. Enviar mensagens do mesmo tipo apenas em 30 segundos (pode ser no py ou com um trigger). Caso a temperatura ou o som estejam frequentemente a acionar alertas (passam o limite e rapidamente voltam ao intervalo normal, e voltam a ultrapassar o limite de novo), limitar o alerta para um intervalo de tempo mais comprido, a não ser que os valores sejam mais elevados do que os valores do último alerta.



1.6 Tratamento de número de marsamis numa sala (obter pontuação)

Aqui devem explicar como funciona o mecanismo que detecta que o número de marsamis mos odd é (ou vai ser) igual ao número de marsamis even. Como detecta, e o que desencadeia.

```
Verificar se marsami é odd ou even pelo número.  
Atualizar OcupacaoLabirinto a cada movimento.  
Trigger verifica se odd == even, se sim e  
num_gatilhos < 3, aciona o gatilho e incrementa  
num_gatilhos
```

```
adicionar num_gatilhos a OcupacaoLabirinto
```



1.7 Especificação de Store Procedures SQL de apoio à migração e tratamento de dados

Nas secções anteriores foram descritos mecanismos que podem ou não necessitar de recorrer a Store Procedures. É nesta tabela que eles deverão ser listados. Na descrição apenas colocar informação que não seja óbvia.

Nome SP	Argumentos	Muito breve descrição
Criar_utilizador	Email; Nome; Telemovel; DataNascimento; Tipo; idEquipa	Cria um novo utilizador e associa-o a uma equipa existente
Remover_utilizador	Email	Remove um utilizador da base de dados pelo seu email
Alterar_utilizador	Nome; Telemovel; DataNascimento; Tipo; Email	Permite alterar os dados pessoais de um utilizador (não altera PK nem FK)
Criar_simulacao	Descrição; DataHoraInicio; Estado; UtilizadorCriador	Cria uma nova simulação associada ao utilizador criador com estado inicial "pendente"
Alterar_simulacao	Descrição; DataHoraInicio; Estado, idSimulação	Altera os campos editáveis de uma simulação – apenas quem a criou pode alterar e não pode estar em curso
Verificar_Outlier	Valor; TipoSensor; Hora	Verifica se o valor recebido de um sensor é



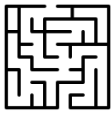
		um outlier com base no histórico recente; se sim, sinaliza-o
Inserir_Alerta	hora; horaEscreita; mensagem; sala; sensor; leitura; tipoALERTA	Insere um registo de alerta na tabela Mensagens com o tipo, sensor, leitura e hora correspondentes



1.8 Especificação de Triggers de apoio à migração e tratamento de dados

Nas secções anteriores foram descritos mecanismos que podem ou não necessitar de recorrer a Triggers. É nesta tabela que eles deverão ser listados. Nas Notas apenas colocar informação que não seja óbvia.

Nome Trigger	Tabela	Tipo de Operação (I,U,D)	Evento (After, Before)	Notas
trg_alerta_som	Som	I	After	Verifica se o valor de som ultrapassa o limite máximo. Se sim, e se não foi inserido alerta nos últimos 30s, insere registo em Mensagens com tipoALERTA='Ruído'
trg_alerta_temperatura	Temperatura	I	After	Verifica se a temperatura ultrapassa o máximo ou o mínimo. Se sim, e se não foi inserido alerta nos últimos 30s, insere registo em Mensagens com tipoALERTA='Temperatura'
trg_outlier_temp	Temperatura	I	Before	Se a temperatura subir ou descer mais que 5°C num instante, marcar como outlier.
trg_atualizar_ocupacao	MedicoesPassagem	I	After	Após cada movimento, decrementa NumeroOdd/NumeroEven na sala de origem e incrementa na sala de destino em OcupacaoLabirinto , consoante o marsami seja odd ou even
trg_verificar_gatilho	OcupacaoLabirinto	U	After	Após atualização da ocupação, verifica se NumeroOdd == NumeroEven e num_gatilhos < 3 . Se sim, insere mensagem de Score em Mensagens e incrementa num_gatilhos
Alterações				retiramos o trg_outlier_som , pq vai ser em python e



				adicionamos o trg_outlier_temp
--	--	--	--	-----------------------------------

1.9 Modelo Relacional

Diagrama relacional completo (modo gráfico). Alterações à base de dados original terão de ser justificadas aqui. Caso seja pertinente podem ser adicionadas comentários a justificar opções pouco óbvias.

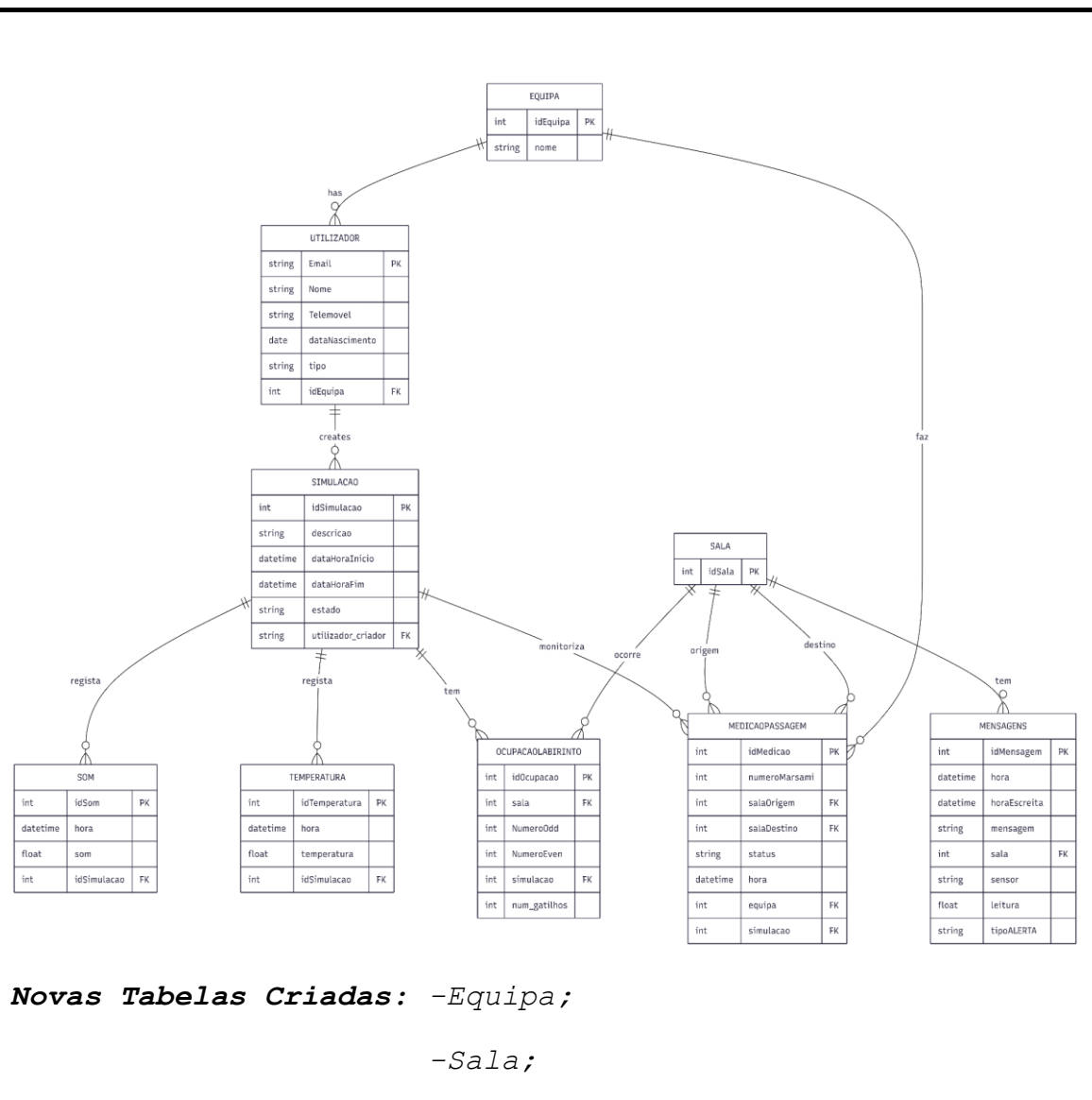




Tabela Simulacao: -Foram incluídos os campos dataHoraFim e estado;

-O campo original Equipa(int) foi removido. Em sua substituição, foi adicionado o utilizador_criador como Chave Estrangeira (FK) ligada à tabela Utilizador,

Tabela dos sensores (Som e Temperatura): -Mudámos o valor das medições (som e temperatura) de varchar(12) para FLOAT;

-Foi adicionado o campo idSimulação (FK) a ambas as tabelas;

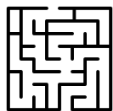
Tabela MedicoesPassagem: -Adicionada nova PK, idOcupação;

-Adicionadas novas FK, equipa, simulação, salaOrigem e salaDestino;

-Foi adicionado um campo num_gatilhos. Isto é para satisfazer a regra que estipula que cada gatilho só pode ser acionado até 3 vezes por sala;

Tabela Mensagens: -O campo Leitura passou a ser FLOAT;

Tabela Utilizador: -O campo Equipa passou a ser FK;



1.10 Utilizadores Base de Dados Mysql

Nesta secção deverá ser explicado de que forma deverá ser feita a manutenção de utilizadores Mysql. Nomeadamente deverá ser indicado, para cada tipo de utilizador, que privilégios ele tem sobre que tabelas e Stored Procedures (todos os SP usados na Aplicação

Tabela	Tipo de Utilizador		
	Administrador	Investigador	Android
Equipa	I/U/D	L	-
Utilizador	I/U/D	L/U	-
Simulacao	I/U/D	I/U/L	L
Sala	I/U/D	L	L
Som	I/U/D	I/L	-
Temperatura	I/U/D	I/L	-
Mensagens	I/U/D	I/L	L
OcupacaoLabirinto	I/U/D	I/U/L	L
MedicoesPassagem	I/U/D	I/L	-
Stored Proc.			
sp_migrar_som	X	X	-
sp_migrar_temperatura	X	X	-
sp_migrar_movimento	X	X	-
sp_inserir_alerta	X	X	-
sp_reiniciar_migracao	X	-	-
sp_criar_simulacao	X	X	-
sp_alterar_simulacao	X	X	-
sp_iniciar_simulacao	X	X	-
sp_terminar_simulacao	X	X	-

Em que U=Update, I Insert, D- Delete, L=Leitura, X=Executar SP e - sem permissões.



1.11 Procedimentos Manutenção da Aplicação

Nesta secção deverão ser listados os SP para a manutenção de utilizadores e jogos (apenas os obrigatórios)

Nome SP	Argumentos	Muito breve descrição
Cria_rut ilizador	email, nome, telemovel, dataNascimento, tipo, idEquipa	Cria um novo utilizador e associa-o a uma equipa
Remover_u tilizador	email	Remove um utilizador pelo email
Alterar_u tilizador	email, nome, telemovel, dataNascimento	Permite ao próprio utilizador alterar os seus dados pessoais (não altera PK nem FK)
Criar_jog o	descricao, utilizador_criador	Cria uma nova simulação com estado "pendente"
Alterar_j ogo	idSimulacao, descricao	Altera campos editáveis de uma simulação – apenas quem a criou pode alterar e não pode estar em curso
Iniciar_j ogo	idSimulacao	Muda o estado da simulação para "em curso" e regista dataHoraInicio
Terminar_ jogo	idSimulacao	Muda o estado para "terminada"



		e regista dataHoraFim
Reiniciar _migracao	idSimulacao	Reinicia o processo de migração em caso de falha – usado pelo administrador



1.12 Eventos de suporte à aplicação (caso existam)

Deverão ser indicados todos os eventos relevantes para o processo de migração, eventos do Windows e do Mysql. Por eventos entende-se as tarefas do Windows ou eventos do Mysql

Nome Evento	Local Execução (Mysql/Windows)	Muito breve descrição



1.13 Consulta por HTML/PHP

Desenhar o layout dos formulários pretendidos, se relevante colocar texto a explicar a funcionalidade pretendida. Formulários:

Fazer login

Criar (ou selecionar de uma lista de jogos um para alterar) um jogo e, para esse jogo, editar os valores associados. Quando está a alterar não pode alterar valores de chaves estrangeiras e primárias.

Indicar para cada botão qual o SP que deverá ser executado

Login:

User

Password

Login

Criar um jogo:

Tipo de Jogo

Jogo 1

Número Salas

Número Equipas

Temperatura Máxima

°C

Temperatura Mínima

°C

Número Passagens

Criar



Editar jogo:

Nome do jogo

Temperatura mínima

Temperatura mínima

Temperatura mínima

Temperatura mínima

Número de passagens

Salvar alterações

2 Implementação

. Aqui vão falar da implementação que fizeram. A implementação é o “best of”, ou seja, o que **agora** acham que é a melhor solução, com base em tudo o que aprenderam (com as vossas experiências, com as ideias do outro grupo, discussões com o professor, google, chatgpt, etc), no limite podem não seguir nada do que tinham especificado no documento que entregaram ao outro grupo.

2.1 Coleções a criar em cada uma das réplicas do Mongo

Versão	Número Coleções
Especificação inicial	3
Recebida outro grupo	
<u>Implementada</u>	3

Justificação da escolha



Texto justificativo da opção final. Se for idêntica à inicial não preencher

Para cada coleção **implementada** exemplifica um documento

Coleção : Motion

```
{
  _id
  69fdf54bc07b5465e0f4d441
  Player
  13
  Marsami
  9
  RoomOrigin
  0
  RoomDestiny
  0
  Status
  2
  migrated
  true
  published
  true
}
```

Coleção : Temperature

```
{
  _id
  69fdf542c07b5465e0f4d43c
  Player
  13
  Hour
  "2026-05-08 15:37:53.845636"
  Temperature
  31.32369012410015
  migrated
  true
  outlier
  false
  published
  true
}
```



}

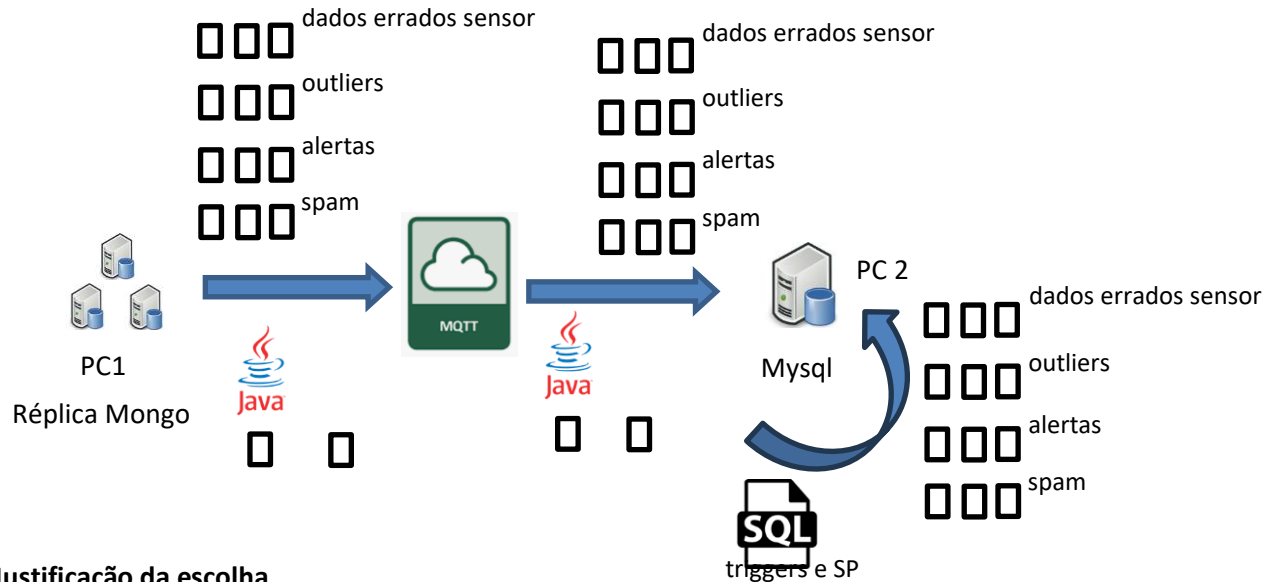
Coleção : Sound

```
{
  _id
  69fdf53fc07b5465e0f4d43a
  Player
  13
  Hour
  "2026-05-08 15:37:50.532278"
  Sound
  10.404
  migrated
  true
  outlier
  false
  published
  true
}
```



2.2 Descrição Geral do Procedimento de Mongo Para Mysql

Na checkbox da esquerda indicam o que especificaram inicialmente, na do meio a especificação do outro grupo e na da direita a vossa implementação.



Justificação da escolha

Texto justificativo da opção final. O que for idêntico à inicial não preencher

Versão	Periodicidade vai buscar Mongo	Como evitam enviar duas vezes para MQTT	Número Threads	QOS
Especificação inicial	1 segundo	Campo booleano "migrated" quando é enviado do mongo para o mqtt	3	1
Recebida outro grupo				
Implementada	1 segundo	Campo booleano "published" quando é enviado do mongo para o mqtt. Campo booleano	3	2



		"migrated" quando chega ao sql		
--	--	-----------------------------------	--	--

Justificação da escolha

Alterámos o QOS para termos a garantia de que o mongo recebia todos os dados, com o 1 não era possível. Adicionámos um campo booleano novo às mensagens no mongo, para também termos a garantia de que o sql recebe os dados todos, se o pc2 estiver em baixo quando o pc1 está a correr procura todos os dados com published=true e migrated=false

Nas próximas quatro secções devem, na descrição, resumidamente descrever num texto esborçado e legível, a forma como foi implementada. Têm de ficar muito explicitamente indicado a o que não resultou da especificação inicial (cor preta), o que foi aproveitado da especificação que receberam de outro grupo (cor azul) e o que resultou de ideias posteriores vossas (cor verde). Na justificação sejam muito objectivos a explicar a razão de terem alterado a especificação inicial

Por exemplo (não necessariamente correto), para deteção de valores anómalos:

1 Descrição

Tratámos dados anómalos resultantes de formatos de datas inválidos (por exemplo, meses com mais de 31 dias), e também considerámos sons negativos como anómalos. Os dados anómalos foram assinalados na coleção como anómalos em vez de serem apagados

2 Justificação das alterações caso tenha havido

Tínhamo-nos esquecido dos valores negativos do som e achamos que pode ser útil alguém querer consultar os valores anómalos posteriormente.



2.3 Tratamento de dados anómalos (valores de sensores errados)

1 Descrição

Marcar como anómalo, no python. Situações- Som negativo; °C > 150? e °C < -50? (apenas exemplo estes valor podemos ver mais tarde concretamente); Som acima de 100db (o mesmo q os ultimos parenteses); Valores impossíveis.

Verificamos se o som e temperatura recebidos em cada mensagem se desviam da média dos valores recentes mais do que um desvio padrão definido, e caso isso se verifique, o dado é assinalado como outlier (tratamos anómalos como outliers). No computador 2, os outliers são extraídos e guardados numa tabela SQL à parte, de modo a não serem utilizados.

2 Justificação das alterações caso tenham havido

Escolhemos verificar o desvio dos dados recentes em vez de ter valores limites definidos porque constitui uma situação mais realista. Se o som ou a temperatura forem valores plausíveis, mas estiverem completamente fora do comum dos valores mais recentes, então podemos concluir que não são verdadeiros. Para além disso, guardados estes dados numa tabela SQL diferente do resto dos dados para podermos ter acesso a todos os dados “falsos” que recebemos, mas de modo a que eles não interfiram em nada.



2.4 Tratamento de outliers de ruído

1 Descrição

Fazer a verificação dos 2 valores seguintes após um pico para confirmar se era outlier, se o valor continuasse dentro do range especificado não era outlier, caso contrário era. Consideramos outliers os picos que hajam de 5db acima da média dos 5 últimos valores, são marcados como outliers, e são colocados, não na tabela do som, mas sim numa tabela chamada "SomOutlier"

2 Justificação das alterações caso tenham havido

Originalmente tivemos aquela ideia porque os picos de som podiam ser gritos ou bater palmas, algo desse gênero. Mas, chegámos à conclusão de que para este projeto isso era algo insignificativo então fizémos igual à temperatura.



2.5 Tratamento de outliers de temperatura

1 Descrição

Utilizámos um trigger para detetar se a temperatura subia mais de 5°C num instante. Consideramos outliers os picos que hajam de 5°C acima da média dos 5 últimos valores, são marcados como outliers, e são colocados, não na tabela da temperatura, mas sim numa tabela chamada "TemperaturaOutlier"

2 Justificação das alterações caso tenham havido

Consideramos que uma média de valores recentes se adequa mais do que ter um valor hardcoded permanente, já que um valor de temperatura que se desvie muito dos últimos valores de temperatura não é possível, logo consideramos um outlier que não tem significado



Tratamento de Alertas de Som

1 Descrição

Utilizámos um trigger que detetava quando o som chegava a um certo limite e mandava mensagem para a tabela “Mensagens”, para o mecanismo de anti-spam só iríamos enviar alertas com 30s de intervalo.

Verificamos a cada mensagem de som, se ela passar o warn_noise (calculado como 80% do intervalo entre o valor normal e o limite máximo, ambos carregados da base de dados cloud no arranque) é inserido na tabela “Mensagens”, com uma flag a identificar a mensagem. Para o mecanismo de anti-spam enviamos alertas iguais com intervalo de 10s.

2 Justificação das alterações caso tenham havido

Alterámos a lógica porque achámos a implementação no python melhor e mais fácil de implementar e poderíamos ter uma lógica mais complexa, algo que o SQL não nos permitiria



2.6 Tratamento de Alertas de Temperatura

1 Descrição

Utilizámos um trigger que detetava quando a temperatura chegava perto do limite, tanto superior como inferior, e mandava mensagem para a tabela “Mensagens”, para o mecanismo de anti-spam só iríamos enviar alertas com 30s de intervalo. *Verificamos a cada mensagem de som, se ela ultrapassar o warn_high_temp (temperatura a aproximar-se do máximo) ou o warn_low_temp (temperatura a aproximar-se do mínimo) é inserido na tabela “Mensagens”, com uma flag a identificar a mensagem. Os avisos são calculados de forma idêntica aos alertas do som. Para o mecanismo de anti-spam enviamos alertas iguais com intervalo de 10s.*

2 Justificação das alterações caso tenham havido

Alterámos a lógica porque achámos a implementação no python melhor e mais fácil de implementar e poderíamos ter uma lógica mais complexa, algo que o SQL não nos permitiria



2.7 Tratamento de número de marsamis numa sala (obter pontuação)

1 Descrição

Utilizámos um trigger que detetava quando o número de marsamis odd era igual ao even, e se fosse e o `num_gatilhos` < 3, aciona o gatilho e incrementa `num_gatilhos` (adicionar `num_gatilhos` a `OcupacaoLabirinto`).

Mudámos completamente a lógica, temos um jogador `player.py` que está permanentemente a correr (como o `computer1.py` e `computer2.py`), que deteta todos os movimentos dos marsamis entre as salas, e calcula o valor de odd e even em cada sala. Caso sejam iguais, fecha as portas e envia o gatilho (até um máximo de 3), e caso tenhamos ultrapassado o limite de gatilhos, abre as portas, maximizando assim o número possível de pontos.

Os marsamis apenas são pontuados a partir de 1 marsami na sala (se a sala tiver 0 odd e 0 even não conta, mesmo tendo tecnicamente o mesmo número de ambos). Os movimentos que têm como origem ou destino a sala 0 não são contabilizados.

2 Justificação das alterações caso tenham havido

A nossa lógica anterior utilizando um trigger teria de ser extremamente complexa para conseguir maximizar os pontos do mesmo modo que a implementação de jogador (`player.py`) que está permanentemente a correr e a verificar todos os movimentos dos marsamis, correndo paralelamente aos computadores 1 e 2.



2.8 Implementação de Stored Procedures SQL de apoio à migração e tratamento de dados

É nesta tabela que deverão ser listados os SP que implementam mecanismos anteriores. Na quarta coluna têm de colocar um dos seguintes símbolos:

= - igual ao especificado

A(og) – Alterado com base em ideia de outro grupo

A(ni) – Alterado com base em novas ideias

N(og) - Novo com base em ideia de outro grupo

N(ni) – Novo com base em ideias novas

A – Alterado com base em novas ideias

Nome SP	Argumentos	Descrição	
Inserir_Alerta	hora, horaEscreita, mensagem, sala, sensor, leitura, tipoALERTA	Insere um registo de alerta na tabela Mensagens. Chamado pelos triggers de som e temperatura	=
Reiniciar_migracao	idSimulacao	Limpa dados migrados a meio e repõe simulação a 'pendente'. Usado só pelo administrador	=
Verificar_Outlier	valor, tipoSensor, idSimulacao, OUT is_outlier	Verifica se valor é outlier com Z-Score dos últimos 5 valores. Camada extra além do Python	N(ni)

Texto justificativo da opção final. O que for idêntico à inicial não preencher



Os SPs `sp_migrar_som`, `sp_migrar_temperatura` e `sp_migrar_movimento` que estavam na especificação inicial foram removidos porque a migração passou a ser feita inteiramente pelo Python, não sendo necessários SPs para esse efeito.

O SP `Verificar_Outlier` é novo, não estava na especificação inicial. Surgiu durante a implementação como camada extra de segurança além do tratamento de outliers já feito em Python, usando Z-Score dos últimos 5 valores.



2.9 Implementação de Triggers

É nesta tabela que deverão ser listados os triggers Na sexta coluna têm de colocar um dos seguintes símbolos:

= - igual ao especificado

A(og) - Alterado com base em ideia de outro grupo

A(ni) - Alterado com base em novas ideias

N(og) - Novo com base em ideia de outro grupo

N(ni) - Novo com base em ideias novas

A - Alterado com base em novas ideias

					de origem e incrementa na sala de destino. salaOrigem=0 significa largada inicial, salaDestino=0 significa marsami preso	=
--	--	--	--	--	--	---

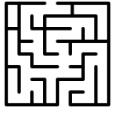
Texto justificativo da opção final. O que for idêntico à inicial não preencher

O trigger trg_outlier_som foi removido — a lógica de deteção de outliers de som ficou toda no Python do computer2 usando Z-Score, sendo mais eficiente tratar isso antes de inserir na base de dados.

O trigger trg_verificar_gatilho foi removido conforme indicação do professor — a lógica de verificar se odd==even e disparar o atuador de score ficou no Python, que consulta a tabela OcupacaoLabirinto a cada segundo e envia a mensagem ao servidor quando a condição se verifica.

O trigger trg_outlier_temperatura foi adaptado — em vez de rejeitar o INSERT marca o registo com anomalo=1 para ser possível consultar esses valores posteriormente se necessário.

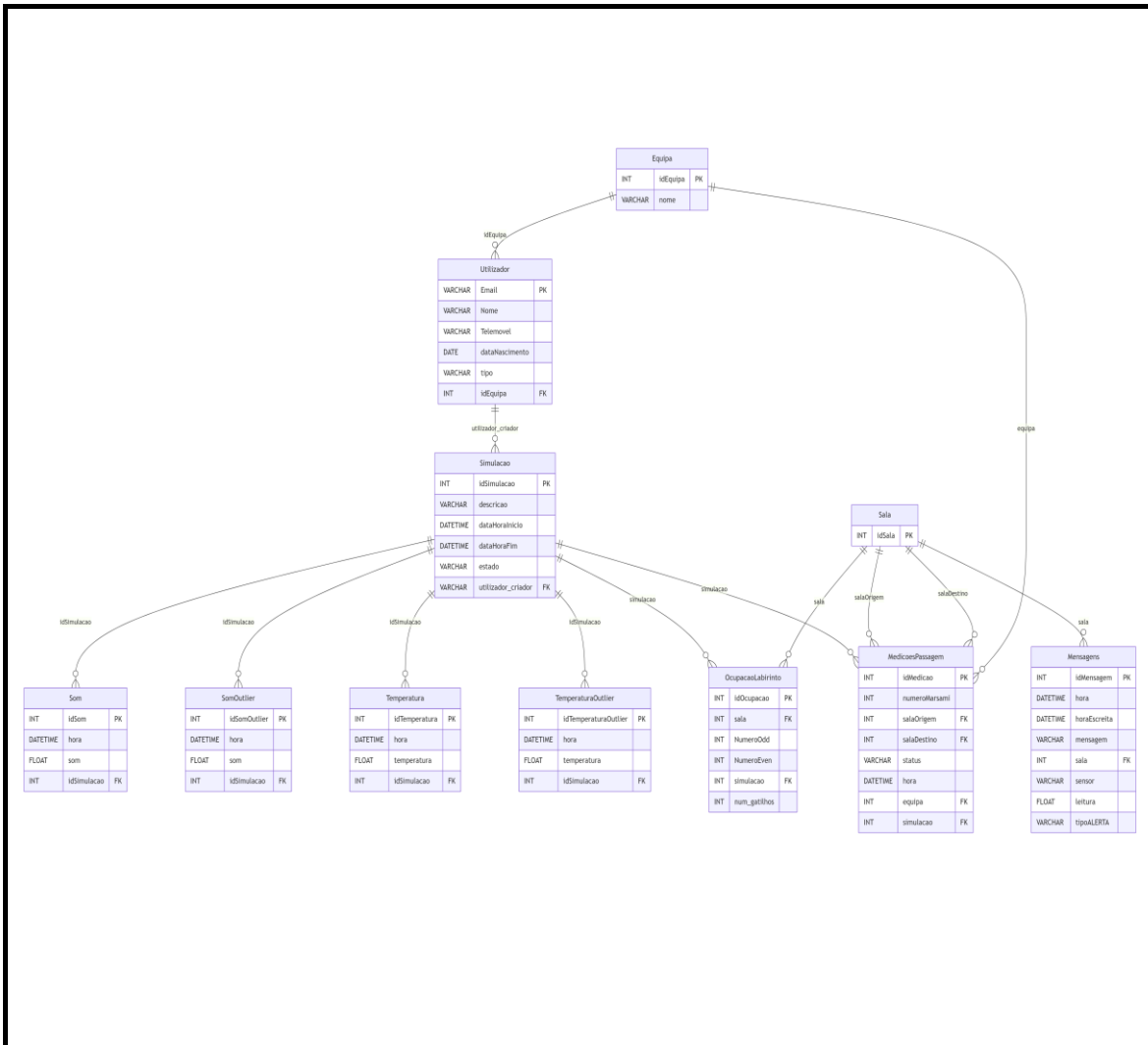
Os triggers de alerta foram mantidos em SQL em vez de Python para centralizar essa lógica na base de dados, sendo ativados automaticamente a cada INSERT sem depender do script Python estar a correr.





1.1 Modelo Relacional

Diagrama relacional completo implementado. Assinalar a azul alterações derivadas de outro grupo e a verde alterações novas.



Adicionámos 2 tabelas para os outliers, porque como nós não queremos eliminar dados nenhuns, assim temos os dados corretos todos juntos e os errados também todos juntos



1.2 Utilizadores Base de Dados Mysql

Nesta secção deverão ser indicados os utilizadores e perfis implementados (têm de constar todos os SP usados). Assinalar a azul alterações derivadas de outro grupo e a verde alterações novas.

Tabela	Tipo de Utilizador		
	Administrador	Investigador	Andorid
Equipa	I/U/D	L	-
Utilizador	I/U/D	L/U	-
Simulacao	I/U/d	I/U/L	L
Sala	I/U/D	L	L
Som	I/U/D	I/L	-
Temperatura	I/U/D	I/L	-
Mensagens	I/U/D	I/L	L
OcupacaoLaboratorio	I/U/D	I/U/L	L
MedicoesPassagem	I/U/D	I/L	-
Stored Proc.			
Inserir_Alerta	X	X	-
Reiniciar_migracao	X	-	-
Criar_simulacao	X	X	-
Alterar_simulacao	X	X	-



Iniciar_simulacao	X	X	-
Terminar_simulacao	X	X	-
Criar_utilizador	X	-	-
Remover_utilizador	X	-	-
Alterar_utilizador	X	X	-
Verificar_Outlier	X	X	-

Em que U=Update, I Insert, D- Delete, L=Leitura, X=Executar SP e - sem permissões.

Texto justificativo da opção final. O que for idêntico à inicial não preencher

Os SPs de migração (**sp_migrar_som**, **sp_migrar_temperatura**, **sp_migrar_movimento**) foram removidos porque a migração passou a ser feita inteiramente pelo Python, não sendo necessários SPs para esse efeito.

Foi adicionado o **SP Verificar_Outlier** como novo, surgido da necessidade de ter uma camada extra de verificação de outliers além do Python.

1.3 Procedimentos Manutenção da Aplicação

É nesta tabela que deverão ser listados os SP que implementam mecanismos anteriores. Na quarta coluna têm de colocar um dos seguintes símbolos:

= - igual ao especificado

A(og) – Alterado com base em ideia de outro grupo

A(ni) – Alterado com base em novas ideias

N(og) - Novo com base em ideia de outro grupo

N(ni) – Novo com base em ideia novas

A – Alterado com base em novas ideias



Nome SP	Argumentos	Descrição	
Inserir_Alerta	hora, horaEscreita, mensagem, sala, sensor, leitura, tipoALERTA	Insere um registo de alerta na tabela Mensagens. Chamado pelos triggers de som e temperatura	=
Reiniciar_migracao	idSimulacao	Limpa dados migrados a meio e repõe simulação a pendente	=
Criar_simulacao	descricao, utilizadorCriador, limSom, limTemperaturaMax, limTemperaturaMin	Cria simulação com estado pendente	A(ni)
Alterar_simulacao	idSimulacao, descricao, utilizadorCriador, limSom, limTemperaturaMax, limTemperaturaMin	Altera campos editáveis, só o criador pode alterar e não pode estar ativa	A(ni)
Iniciar_simulacao	idSimulacao	Muda estado para ativo e regista dataHoraInicio	A(ni)



Terminar_simulacao	idSimulacao	Muda estado para terminada e regista dataHoraFim	=
Criar_utilizador	email, nome, telemovel, dataNascimento, tipo, idEquipa	Cria novo utilizador associado a uma equipa	=
Remover_utilizador	email	Remove utilizador pelo email	=
Alterar_utilizador	email, nome, telemovel, dataNascimento, tipo	Altera dados pessoais, não altera PK nem FK	=
Verificar_Outlier	valor, tipoSensor, idSimulacao, OUT is_outlier	Verifica se valor é outlier com Z-Score dos últimos 5 valores. Camada extra além do Python	N(ni)

Texto justificativo da opção final. O que for idêntico à inicial não preencher

Os **SPs de criar e alterar simulação** foram alterados para incluir os limites de som e temperatura como argumentos, porque esses limites passaram a estar na tabela Simulacao para suportar os alertas.

O **SP Iniciar_simulacao** usa o estado 'ativo' em vez de 'em curso' para ficar consistente com o Python do computer2.



1.4 Eventos de suporte à aplicação (caso existam)

É nesta tabela que eles deverão ser listados todos os eventos relevantes para o processo de migração Na quarta coluna têm de colocar um dos seguintes símbolos:

= - igual ao especificado

A(og) - Alterado com base em ideia de outro grupo

A(ni) - Alterado com base em novas ideias

N(og) - Novo com base em ideia de outro grupo

N(ni) - Novo com base em ideias novas

A - Alterado com base em novas ideias

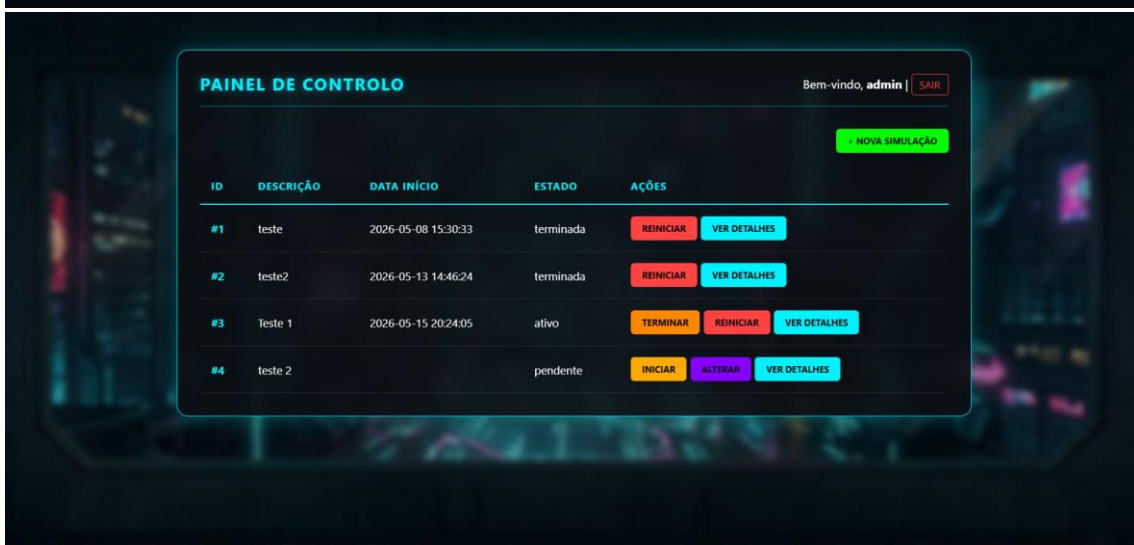
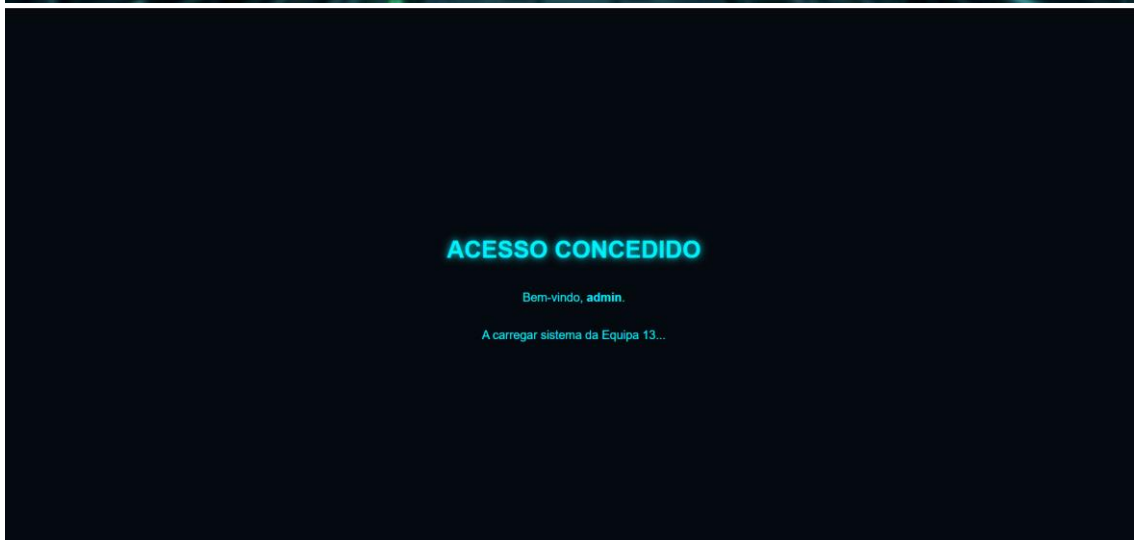
Nome Evento	Local Execução (Mysql/Windows)	Muito breve descrição

Texto justificativo da opção final. O que for idêntico à não preencher

Não foram necessários eventos MySQL porque a lógica de polling é feita pelo Python com `time.sleep(1)` nas threads, dispensando eventos agendados na base de dados.



1.5 PrintScreen dos formulários HTML implementados





NOVA SIMULAÇÃO

Descrição do Labirinto

PARÂMETROS DE TEMPERATURA (°C)

Máximo: 30.0 Mínimo: 15.0

PARÂMETROS DE SOM (dB)

Limite Máximo: 80.0

LANÇAR SIMULAÇÃO

VOLTAR AO PAINEL

ALTERAR SIMULAÇÃO #4

Insto 2

Descrição

PARÂMETROS DE TEMPERATURA (°C)

Máximo: 30 Mínimo: 15

PARÂMETROS DE SOM (dB)

Limite Máximo: 80

GUARDAR ALTERAÇÕES

VOLTAR AO PAINEL





1.6 PrintScreen do formulários Android com dados

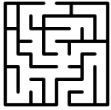






Anexo Código SQL

```
CREATE TABLE Equipa (  
    idEquipa INT NOT NULL AUTO_INCREMENT,  
    nome VARCHAR(255) NOT NULL,  
    PRIMARY KEY (idEquipa)  
);  
  
CREATE TABLE Utilizador (  
    Email VARCHAR(255) NOT NULL,  
    Nome VARCHAR(255) NOT NULL,  
    Telemovel VARCHAR(9) NOT NULL,  
    dataNascimento DATE NOT NULL,  
    tipo VARCHAR(255) NOT NULL,  
    idEquipa INT NOT NULL,  
    PRIMARY KEY (Email),  
    FOREIGN KEY (idEquipa) REFERENCES Equipa(idEquipa)  
);  
  
CREATE TABLE Simulacao (  
    idSimulacao INT NOT NULL AUTO_INCREMENT,  
    descricao VARCHAR(255) NOT NULL,  
    dataHoraInicio DATETIME NOT NULL,  
    dataHoraFim DATETIME,  
    estado VARCHAR(255) NOT NULL,  
    utilizador_criador VARCHAR(255) NOT NULL,  
    PRIMARY KEY (idSimulacao),  
    FOREIGN KEY (utilizador_criador) REFERENCES Utilizador(Email)  
);  
  
CREATE TABLE Sala (  
    idSala INT NOT NULL AUTO_INCREMENT,  
    PRIMARY KEY (idSala)  
);  
  
CREATE TABLE Som (  
    idSom INT NOT NULL AUTO_INCREMENT,  
    hora DATETIME NOT NULL,  
    som FLOAT NOT NULL,  
    idSimulacao INT NOT NULL,  
    PRIMARY KEY (idSom),
```



```
FOREIGN KEY (idSimulacao) REFERENCES Simulacao(idSimulacao)
);

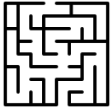
CREATE TABLE SomOutlier (
    idSomOutlier INT NOT NULL AUTO_INCREMENT,
    hora DATETIME NOT NULL,
    som FLOAT NOT NULL,
    idSimulacao INT NOT NULL,
    PRIMARY KEY (idSomOutlier),
    FOREIGN KEY (idSimulacao) REFERENCES Simulacao(idSimulacao)
);

CREATE TABLE Temperatura (
    idTemperatura INT NOT NULL AUTO_INCREMENT,
    hora DATETIME NOT NULL,
    temperatura FLOAT NOT NULL,
    idSimulacao INT NOT NULL,
    PRIMARY KEY (idTemperatura),
    FOREIGN KEY (idSimulacao) REFERENCES Simulacao(idSimulacao)
);

CREATE TABLE TemperaturaOutlier (
    idTemperaturaOutlier INT NOT NULL AUTO_INCREMENT,
    hora DATETIME NOT NULL,
    temperatura FLOAT NOT NULL,
    idSimulacao INT NOT NULL,
    PRIMARY KEY (idTemperaturaOutlier),
    FOREIGN KEY (idSimulacao) REFERENCES Simulacao(idSimulacao)
);

CREATE TABLE Mensagens (
    idMensagem INT AUTO_INCREMENT,
    hora DATETIME,
    horaEscreita DATETIME,
    mensagem VARCHAR(255),
    sala INT,
    sensor VARCHAR(255),
    leitura FLOAT,
    tipoALERTA VARCHAR(255),
    PRIMARY KEY (idMensagem),
    FOREIGN KEY (sala) REFERENCES Sala(idSala)
);

CREATE TABLE OcupacaoLabirinto (
    idOcupacao INT NOT NULL AUTO_INCREMENT,
    sala INT NOT NULL,
```



```
NumeroOdd INT NOT NULL,
NumeroEven INT NOT NULL,
simulacao INT NOT NULL,
num_gatilhos INT NOT NULL,
PRIMARY KEY (idOcupacao),
FOREIGN KEY (sala) REFERENCES Sala(idSala),
FOREIGN KEY (simulacao) REFERENCES Simulacao(idSimulacao)
);

CREATE TABLE MedicoesPassagem (
  idMedicao INT NOT NULL AUTO_INCREMENT,
  numeroMarsami INT NOT NULL,
  salaOrigem INT NOT NULL,
  salaDestino INT NOT NULL,
  status VARCHAR(255) NOT NULL,
  hora DATETIME NOT NULL,
  equipa INT NOT NULL,
  simulacao INT NOT NULL,
  PRIMARY KEY (idMedicao),
  FOREIGN KEY (salaOrigem) REFERENCES Sala(idSala),
  FOREIGN KEY (salaDestino) REFERENCES Sala(idSala),
  FOREIGN KEY (equipa) REFERENCES Equipa(idEquipa),
  FOREIGN KEY (simulacao) REFERENCES Simulacao(idSimulacao)
);

CREATE USER 'Administrador'@'%' IDENTIFIED BY 'password';

GRANT SELECT, INSERT, UPDATE, DELETE ON pisis.Utilizador, pisis.Equipa,
pisis.Simulacao, pisis.Sala, pisis.Som, pisis.SomOutlier,
pisis.Temperatura, pisis.TemperaturaOutlier, pisis.Mensagens,
pisis.OcupacaoLabirinto, pisis.MedicoesPassagem TO 'Administrador'@'%';

CREATE USER 'Investigador'@'%' IDENTIFIED BY 'Investig0';

GRANT SELECT, INSERT, UPDATE ON pisis.Simulacao, pisis.OcupacaoLabirinto
TO 'Investigador'@'%';

GRANT SELECT, INSERT ON pisis.Som, pisis.Temperatura, pisis.Mensagens,
pisis.MedicoesPassagem TO 'Investigador'@'%';

GRANT SELECT, UPDATE ON pisis.Utilizador TO 'Investigador'@'%';

GRANT SELECT ON pisis.Equipa, pisis.Sala TO 'Investigador'@'%';
```



```
CREATE USER 'Android'@'%' IDENTIFIED BY 'Iphone';  
  
GRANT SELECT ON pisid.Simulacao, pisid.Sala, pisid.Mensagens,  
pisid.OcupacaoLabirinto TO 'Android'@'%';  
  
FLUSH PRIVILEGES;
```



Código de Triggers implementados

1. Nome Trigger: trg_atualizar_ocupacao

Depois de inserir na tabela MedicoesPassagem, atualiza a contagem de marsamis odd/even na tabela OcupacaoLabirinto.

Decrementa na sala de origem e incrementa na sala de destino.

RoomOrigin = 0 -> largada inicial, não decrementa.

RoomDestiny = 0 -> marsami preso, não incrementa.

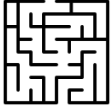
Código

```
DELIMITER $$
DROP TRIGGER IF EXISTS trg_atualizar_ocupacao$$
CREATE TRIGGER trg_atualizar_ocupacao
AFTER INSERT ON MedicoesPassagem
FOR EACH ROW
BEGIN
    DECLARE v_tipo VARCHAR(4);

    -- Determinar se o marsami é odd ou even pelo número
    IF MOD(NEW.numeroMarsami, 2) = 1 THEN
        SET v_tipo = 'odd';
    ELSE
        SET v_tipo = 'even';
    END IF;

    -- Decrementar na sala de origem (se não for largada inicial)
    IF NEW.salaOrigem != 0 THEN
        IF v_tipo = 'odd' THEN
            UPDATE OcupacaoLabirinto
            SET NumeroOdd = NumeroOdd - 1
            WHERE sala = NEW.salaOrigem AND simulacao =
NEW.simulacao AND NumeroOdd > 0;
        ELSE
            UPDATE OcupacaoLabirinto
            SET NumeroEven = NumeroEven - 1
            WHERE sala = NEW.salaOrigem AND simulacao =
NEW.simulacao AND NumeroEven > 0;
        END IF;
    END IF;

    -- Incrementar na sala de destino (se marsami não estiver preso)
```



```
IF NEW.salaDestino != 0 THEN
  IF v_tipo = 'odd' THEN
    UPDATE OcupacaoLabirinto
    SET NumeroOdd = NumeroOdd + 1
    WHERE sala = NEW.salaDestino AND simulacao =
NEW.simulacao;
  ELSE
    UPDATE OcupacaoLabirinto
    SET NumeroEven = NumeroEven + 1
    WHERE sala = NEW.salaDestino AND simulacao =
NEW.simulacao;
  END IF;
END IF;

END$$
DELIMITER ;
```



Código Stored Procedures implementados

```
1. Nome SP: Inserir_Alerta
-- Insere um registo de alerta na tabela Mensagens.
-- Chamado pelos triggers trg_alerta_som e
trg_alerta_temperatura.
-----
DELIMITER $$
DROP PROCEDURE IF EXISTS Inserir_Alerta$$
CREATE PROCEDURE Inserir_Alerta(
    IN p_hora DATETIME,
    IN p_horaEscreita DATETIME,
    IN p_mensagem VARCHAR(255),
    IN p_sala INT,
    IN p_sensor VARCHAR(255),
    IN p_leitura FLOAT,
    IN p_tipoALERTA VARCHAR(255)
)
BEGIN
    INSERT INTO Mensagens (hora, horaEscreita, mensagem,
sala, sensor, leitura, tipoALERTA)
    VALUES (p_hora, p_horaEscreita, p_mensagem, p_sala,
p_sensor, p_leitura, p_tipoALERTA);
END$$
DELIMITER ;

-----

2. Nome SP: Criar_utilizador
-- Cria um novo utilizador e associa-o a uma equipa
existente.
-----
DELIMITER $$
DROP PROCEDURE IF EXISTS Criar_utilizador$$
CREATE PROCEDURE Criar_utilizador(
    IN p_email VARCHAR(255),
    IN p_nome VARCHAR(255),
    IN p_telemovei VARCHAR(9),
    IN p_dataNascimento DATE,
    IN p_tipo VARCHAR(255),
    IN p_idEquipa INT
)
BEGIN
    IF EXISTS (SELECT 1 FROM Utilizador WHERE Email =
p_email) THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Erro: já existe um utilizador
com este email.';
    END IF;
END IF;
```




```
IF NOT EXISTS (SELECT 1 FROM Equipa WHERE idEquipa =
p_idEquipa) THEN
    SIGNAL SQLSTATE '45000'
    SET MESSAGE_TEXT = 'Erro: a equipa indicada não
existe.';
END IF;

INSERT INTO Utilizador (Email, Nome, Telemovel,
dataNascimento, tipo, idEquipa)
VALUES (p_email, p_nome, p telemovel,
p_dataNascimento, p_tipo, p_idEquipa);
END$$
DELIMITER ;

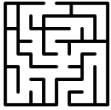
-----

3. Nome SP: Remover_utilizador
-- Remove um utilizador da base de dados pelo seu email.
-----
DELIMITER $$
DROP PROCEDURE IF EXISTS Remover_utilizador$$
CREATE PROCEDURE Remover_utilizador(
    IN p_email VARCHAR(255)
)
BEGIN
    IF NOT EXISTS (SELECT 1 FROM Utilizador WHERE Email =
p_email) THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Erro: utilizador não
encontrado.';
    END IF;

    DELETE FROM Utilizador WHERE Email = p_email;
END$$
DELIMITER ;

-----

4. Nome SP: Alterar_utilizador
-- Permite alterar os dados pessoais de um utilizador.
-- Não altera PK (Email) nem FK (idEquipa).
-----
DELIMITER $$
DROP PROCEDURE IF EXISTS Alterar_utilizador$$
CREATE PROCEDURE Alterar_utilizador(
    IN p_email VARCHAR(255),
    IN p_nome VARCHAR(255),
    IN p telemovel VARCHAR(9),
    IN p_dataNascimento DATE,
    IN p_tipo VARCHAR(255)
)
BEGIN
```



```
IF NOT EXISTS (SELECT 1 FROM Utilizador WHERE Email =
p_email) THEN
    SIGNAL SQLSTATE '45000'
    SET MESSAGE_TEXT = 'Erro: utilizador não
encontrado.';
END IF;

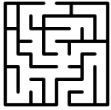
UPDATE Utilizador
SET Nome = p_nome,
    Telemovel = p_telemovei,
    dataNascimento = p_dataNascimento,
    tipo = p_tipo
WHERE Email = p_email;
END$$
DELIMITER ;
```

```
-----
5. Nome SP: Criar_simulacao
-- Cria uma nova simulação com estado inicial 'pendente'.
-- Os limites de som e temperatura podem ser definidos
aqui.
```

```
-----
DELIMITER $$
DROP PROCEDURE IF EXISTS Criar_simulacao$$
CREATE PROCEDURE Criar_simulacao(
    IN p_descricao VARCHAR(255),
    IN p_utilizadorCriador VARCHAR(255),
    IN p_limSom FLOAT,
    IN p_limTemperaturaMax FLOAT,
    IN p_limTemperaturaMin FLOAT
)
BEGIN
    IF NOT EXISTS (SELECT 1 FROM Utilizador WHERE Email =
p_utilizadorCriador) THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Erro: utilizador criador não
encontrado.';
    END IF;

    INSERT INTO Simulacao (descricao, dataHoraInicio,
dataHoraFim, estado, utilizador_criador, limSom,
limTemperaturaMax, limTemperaturaMin)
VALUES (p_descricao, NULL, NULL, 'pendente',
p_utilizadorCriador, p_limSom, p_limTemperaturaMax,
p_limTemperaturaMin);
END$$
DELIMITER ;
```

```
-----
6. Nome SP: Alterar_simulacao
-- Permite alterar campos editáveis de uma simulação.
```



```
-- Só quem criou pode alterar e não pode estar em curso.
-----
DELIMITER $$
DROP PROCEDURE IF EXISTS Alterar_simulacao$$
CREATE PROCEDURE Alterar_simulacao(
    IN p_idSimulacao INT,
    IN p_descricao VARCHAR(255),
    IN p_utilizadorCriador VARCHAR(255),
    IN p_limSom FLOAT,
    IN p_limTemperaturaMax FLOAT,
    IN p_limTemperaturaMin FLOAT
)
BEGIN
    DECLARE v_criador VARCHAR(255);
    DECLARE v_estado VARCHAR(255);

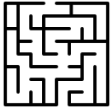
    SELECT utilizador_criador, estado INTO v_criador,
v_estado
    FROM Simulacao
    WHERE idSimulacao = p_idSimulacao;

    IF v_criador IS NULL THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Erro: simulação não
encontrada.';
    END IF;

    IF v_criador != p_utilizadorCriador THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Erro: apenas quem criou a
simulação pode alterá-la.';
    END IF;

    IF v_estado = 'ativo' THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Erro: não é possível alterar
uma simulação em curso.';
    END IF;

    UPDATE Simulacao
    SET descricao = p_descricao,
        limSom = p_limSom,
        limTemperaturaMax = p_limTemperaturaMax,
        limTemperaturaMin = p_limTemperaturaMin
    WHERE idSimulacao = p_idSimulacao;
END$$
DELIMITER ;
-----
7. Nome SP: Iniciar_simulacao
```



```
-- Muda o estado para 'em curso' e regista data/hora de
início.
-----
DELIMITER $$
DROP PROCEDURE IF EXISTS Iniciar_simulacao$$
CREATE PROCEDURE Iniciar_simulacao(
    IN p_idSimulacao INT
)
BEGIN
    DECLARE v_estado VARCHAR(255);

    SELECT estado INTO v_estado
    FROM Simulacao
    WHERE idSimulacao = p_idSimulacao;

    IF v_estado IS NULL THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Erro: simulação não
encontrada.';
    END IF;

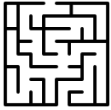
    IF v_estado = 'ativo' THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Erro: simulação já está em
curso.';
    END IF;

    IF v_estado = 'terminada' THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Erro: simulação já foi
terminada.';
    END IF;

    UPDATE Simulacao
    SET estado = 'ativo',
        dataHoraInicio = NOW()
    WHERE idSimulacao = p_idSimulacao;

    DELETE FROM OcupacaoLabirinto WHERE simulacao =
p_idSimulacao;

    INSERT INTO OcupacaoLabirinto (sala, NumeroOdd,
NumeroEven, num_gatilhos, simulacao) VALUES
    (1, 0, 0, 0, p_idSimulacao),
    (2, 0, 0, 0, p_idSimulacao),
    (3, 0, 0, 0, p_idSimulacao),
    (4, 0, 0, 0, p_idSimulacao),
    (5, 0, 0, 0, p_idSimulacao),
    (6, 0, 0, 0, p_idSimulacao),
```



```
(7, 0, 0, 0, p_idSimulacao),
(8, 0, 0, 0, p_idSimulacao),
(9, 0, 0, 0, p_idSimulacao),
(10, 0, 0, 0, p_idSimulacao);
END$$
DELIMITER ;

-----

8. Nome SP: Terminar_simulacao
-- Muda o estado para 'terminada' e regista data/hora de
fim.
-----

DELIMITER $$
DROP PROCEDURE IF EXISTS Terminar_simulacao$$
CREATE PROCEDURE Terminar_simulacao(
    IN p_idSimulacao INT
)
BEGIN
    DECLARE v_estado VARCHAR(255);

    SELECT estado INTO v_estado
    FROM Simulacao
    WHERE idSimulacao = p_idSimulacao;

    IF v_estado IS NULL THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Erro: simulação não
encontrada.';
    END IF;

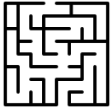
    IF v_estado = 'terminada' THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Erro: simulação já foi
terminada.';
    END IF;

    UPDATE Simulacao
    SET estado = 'terminada',
        dataHoraFim = NOW()
    WHERE idSimulacao = p_idSimulacao;
END$$
DELIMITER ;

-----

9. Nome SP: Reiniciar_migracao
-- Reinicia o processo de migração em caso de falha.
-- Usado exclusivamente pelo administrador.
-- Limpa dados migrados a meio e repõe simulação a
'pendente'.
-----

DELIMITER $$
```



```
DROP PROCEDURE IF EXISTS Reiniciar_migracao$$
CREATE PROCEDURE Reiniciar_migracao(
    IN p_idSimulacao INT
)
BEGIN
    DECLARE v_estado VARCHAR(255);

    SELECT estado INTO v_estado
    FROM Simulacao
    WHERE idSimulacao = p_idSimulacao;

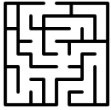
    IF v_estado IS NULL THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Erro: simulação não
encontrada.';
    END IF;

    -- Limpar dados dos sensores desta simulação
    DELETE FROM Som WHERE idSimulacao = p_idSimulacao;
    DELETE FROM SomOutlier WHERE idSimulacao =
p_idSimulacao;
    DELETE FROM Temperatura WHERE idSimulacao =
p_idSimulacao;
    DELETE FROM TemperaturaOutlier WHERE idSimulacao =
p_idSimulacao;
    DELETE FROM MedicoesPassagem WHERE simulacao =
p_idSimulacao;

    -- Repor ocupação do labirinto a zeros
    UPDATE OcupacaoLabirinto
    SET NumeroOdd = 0,
        NumeroEven = 0,
        num_gatilhos = 0
    WHERE simulacao = p_idSimulacao;

    -- Apagar alertas gerados durante a migração falhada
    DELETE FROM Mensagens
    WHERE hora >= (
        SELECT dataHoraInicio FROM Simulacao WHERE
idSimulacao = p_idSimulacao
    );

    -- Voltar simulação a pendente
    UPDATE Simulacao
    SET estado = 'pendente',
        dataHoraInicio = NULL,
        dataHoraFim = NULL
    WHERE idSimulacao = p_idSimulacao;
```



```
END$$
DELIMITER ;

-----
10.      Nome: Verificar_Outlier
-- Verifica se um valor é outlier com base nos últimos 5
valores
-- usando Z-Score. Retorna 1 se outlier, 0 se normal.
-- Camada extra de segurança além do Python.
-- (Removido o anomalo = 0 porque a tabela principal só
tem dados normais agora)
-----

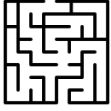
DELIMITER $$
DROP PROCEDURE IF EXISTS Verificar_Outlier$$
CREATE PROCEDURE Verificar_Outlier(
    IN p_valor FLOAT,
    IN p_tipoSensor VARCHAR(255),
    IN p_idSimulacao INT,
    OUT p_is_outlier INT
)
BEGIN
    DECLARE v_media FLOAT;
    DECLARE v_desvio FLOAT;
    DECLARE v_zscore FLOAT;

    SET p_is_outlier = 0;

    IF p_tipoSensor = 'Som' THEN
        SELECT AVG(som), STD(som) INTO v_media, v_desvio
        FROM (
            SELECT som FROM Som
            WHERE idSimulacao = p_idSimulacao
            ORDER BY hora DESC LIMIT 5
        ) AS ultimos;

    ELSEIF p_tipoSensor = 'Temperatura' THEN
        SELECT AVG(temperatura), STD(temperatura) INTO
v_media, v_desvio
        FROM (
            SELECT temperatura FROM Temperatura
            WHERE idSimulacao = p_idSimulacao
            ORDER BY hora DESC LIMIT 5
        ) AS ultimos;
    END IF;

    IF v_media IS NOT NULL AND v_desvio IS NOT NULL AND
v_desvio > 0 THEN
        SET v_zscore = ABS((p_valor - v_media) /
v_desvio);
        IF v_zscore > 3 THEN
```



```
        SET p_is_outlier = 1;  
    END IF;  
END IF;  
  
END$$  
DELIMITER ;
```